CarSharing Dalmine

Iterazione 1

Sistema intelligente di Car Sharing ottimizzato per il polo universitario di Dalmine

Sommario

1. Scopo dell'Iterazione 3

2. Architettura Iniziale del Sistema 4

2.1 Vista Logica – Organizzazione in Layer 4

Data Access Layer 5

Service Layer 5

Business Layer 6

Presentation Layer 6

2.2 Class Diagram – Models 7

2.3 Vista Fisica – Deployment 7

3. Struttura del Progetto e Setup IDE 9

3.1 Albero del Progetto 9

1. Scopo dell'Iterazione

Questa iterazione ha come obiettivo la definizione dell'architettura iniziale del sistema e la predisposizione dell'ambiente di sviluppo. In questa fase verranno implementate la struttura del progetto e l'organizzazione dei componenti secondo l'architettura a layer definita.

Al termine di questa iterazione il sistema sarà composto da:

- **Architettura iniziale documentata**: organizzazione del sistema nei 4 layer (Data Access, Services, Business, Presentation) con relative responsabilità e dipendenze
- **Setup del progetto nell'IDE**: struttura delle cartelle e dei file conforme all'architettura, con classi e metodi stub (vuoti), pronti per essere implementati nelle iterazioni successive

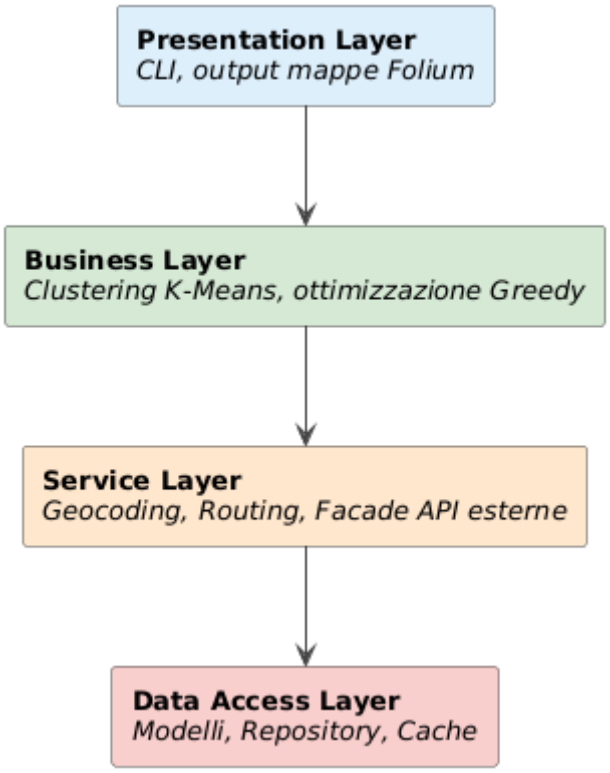
---

2. Architettura Iniziale del Sistema

Questa sezione descrive l'organizzazione strutturale del sistema, definendo la suddivisione in layer dei vari componenti.

2.1 Vista Logica – Organizzazione in Layer

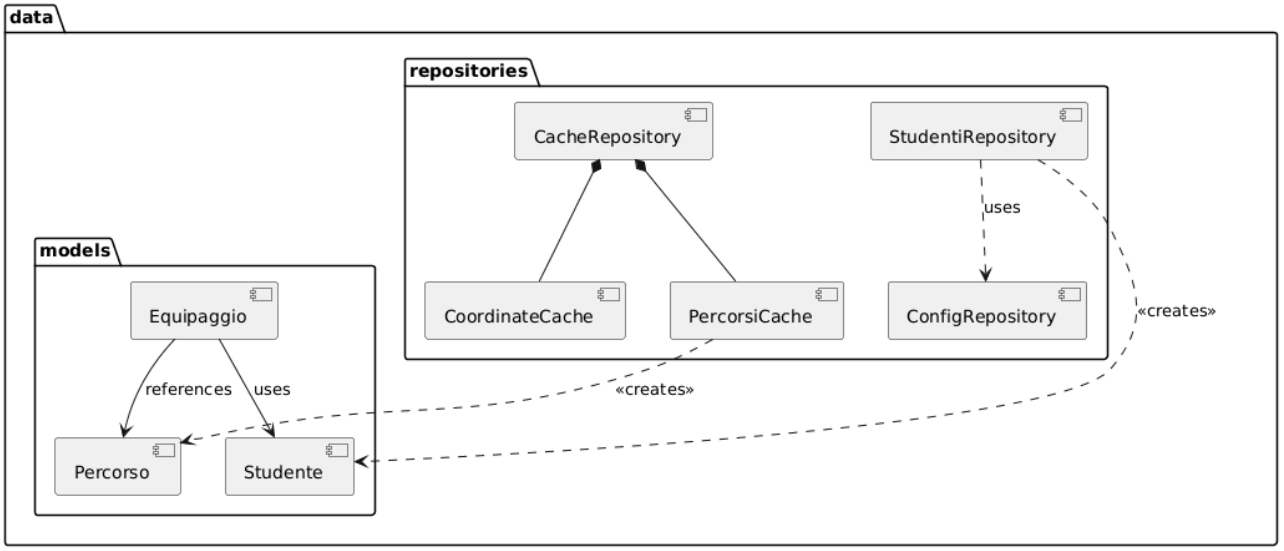
Il sistema è organizzato in **4 layer** con dipendenze unidirezionali rilassate (ogni layer dipende solo dai layer sottostanti):



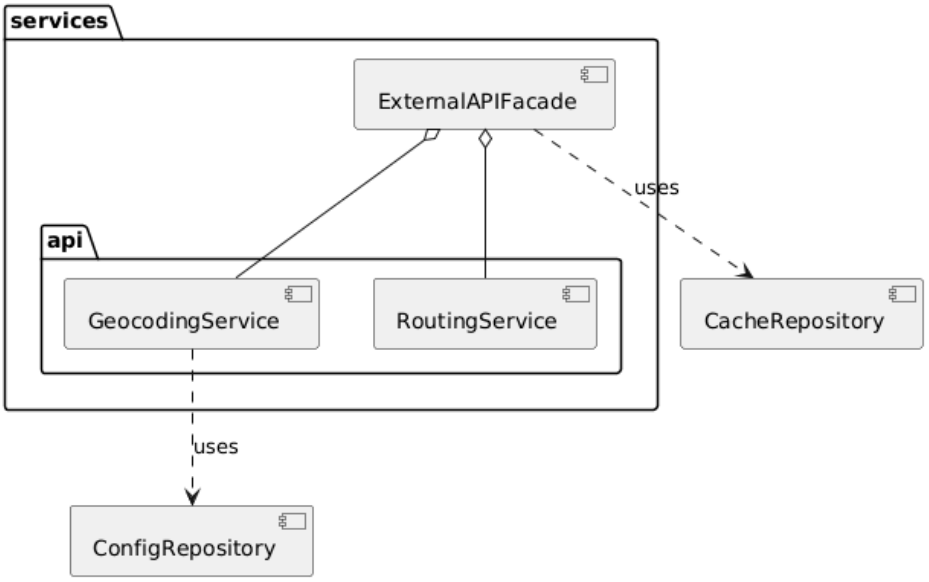
Responsabilità dei layer:

Layer	Responsabilità	Componenti principali
Data Access	Persistenza, modelli di dominio, cache JSON	Studente, Percorso, Equipaggio, StudentiRepository, CacheRepository, ConfigRepository
Services	Integrazione API esterne, geocoding, routing	GeocodingService, RoutingService, ExternalAPIFacade
Business	Logica di ottimizzazione carpooling	ClusteringService (K-Means), GreedyOptimizer, CarpoolingOrchestrator
Presentation	Interfaccia utente e visualizzazione	CLI, MapRenderer (Folium)

Data Access Layer

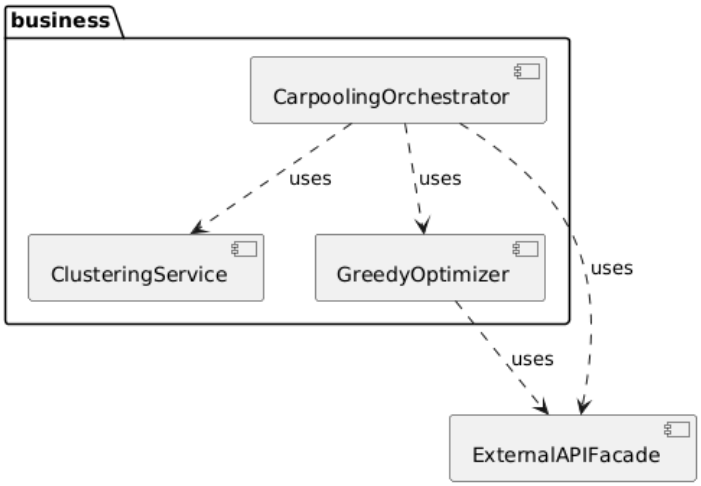


Service Layer

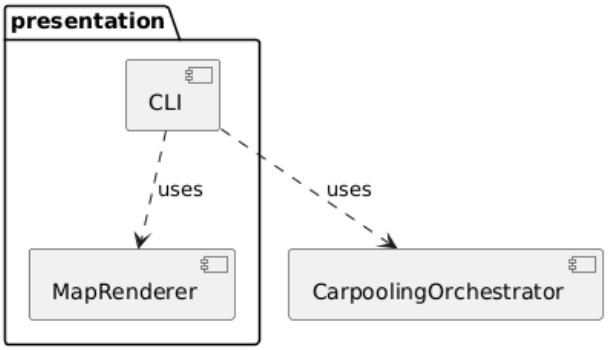


Business Layer

I componenti del Business Layer sono definiti nelle loro interfacce pubbliche, senza implementazione:

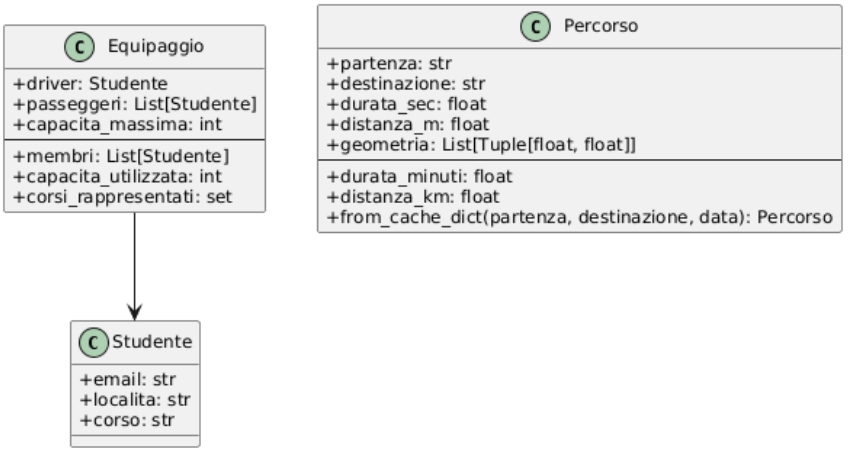


Presentation Layer



2.2 Class Diagram – Models

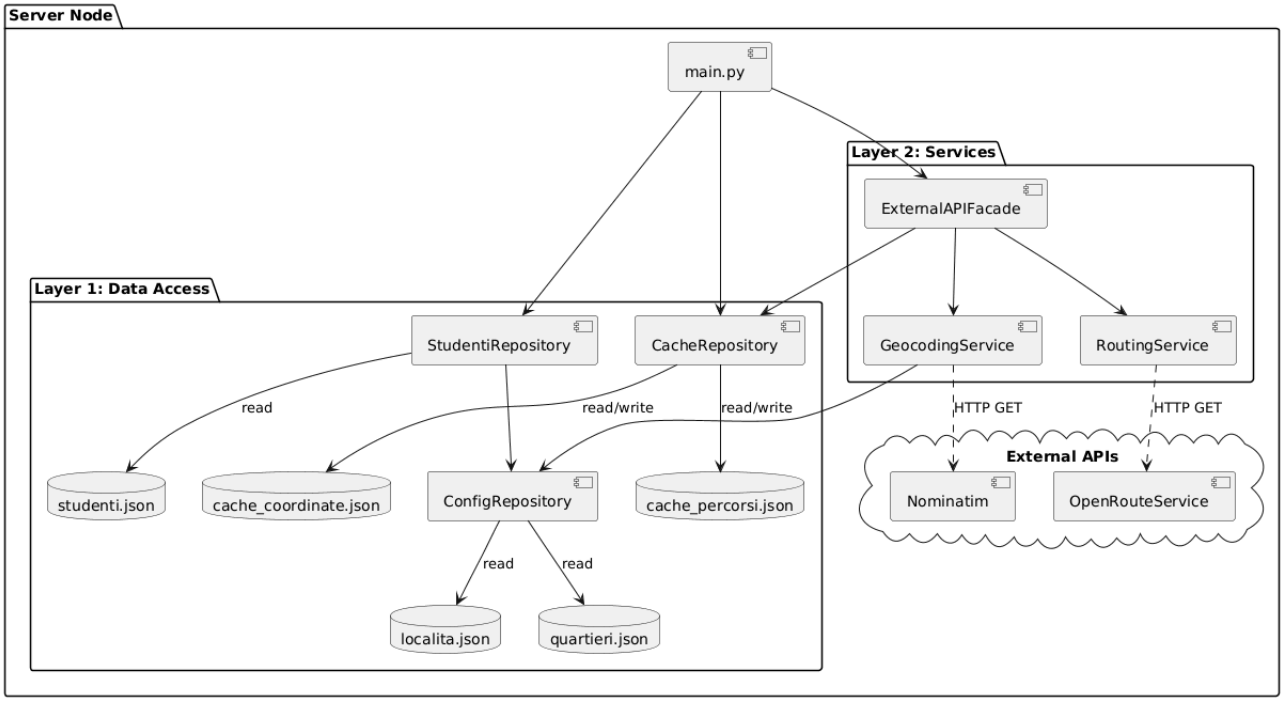
I modelli di dominio rappresentano le entità principali del sistema. Sono definiti come dataclass Python per garantire type safety e immutabilità dei dati.



Descrizione delle entità:

- **Studente**: rappresenta un utente del sistema con email, località di partenza e corso di laurea.
- **Percorso**: rappresenta un percorso stradale reale tra due località, con durata, distanza e geometria (lista di coordinate). Espone proprietà calcolate per la conversione delle unità di misura
- **Equipaggio**: aggrega un driver e zero o più passeggeri, con un limite di capacità massima. Espone proprietà calcolate per la composizione del gruppo.

2.3 Vista Fisica – Deployment



La deployment view mostra un'architettura **client-server semplificata**: il backend Python gira in locale sulla macchina dello studente, comunica con le API esterne HeiGIT (Nominatim + OpenRouteService) via HTTP, e persiste i dati su filesystem locale tramite file JSON.

---

3. Struttura del Progetto e Setup IDE

Questo capitolo mostra la struttura del progetto così come si presenta nell'IDE al termine dell'Iterazione 1: le cartelle e i file sono organizzati in conformità con l'architettura a 4 layer, con i layer Data Access e Services già implementati, e i layer Business e Presentation presenti come stub.

3.1 Albero del Progetto

```
carsharing-dalmine/
│
├── .env
├── .gitignore
├── requirements.txt
├── README.md
├── docs/
│
├── data/
│   ├── input/
│   │   └── studenti.json
│   ├── config/
│   │   ├── localita.json
│   │   └── quartieri.json
│   └── cache/
│       ├── cache_coordinate.json
│       └── cache_percorsi.json
│
├── src/
│   ├── __init__.py
│   │
│   ├── data/
│   │   ├── __init__.py
│   │   ├── models/
│   │   │   ├── __init__.py
│   │   │   ├── studente.py
│   │   │   ├── percorso.py
│   │   │   └── equipaggio.py
│   │   └── repositories/
│   │       ├── __init__.py
│   │       ├── studenti_repository.py
│   │       ├── cache_repository.py
│   │       └── config_repository.py
│   │
│   ├── services/
│   │   ├── __init__.py
│   │   ├── external_api_facade.py
│   │   └── api/
│   │       ├── __init__.py
│   │       ├── geocoding_service.py
│   │       └── routing_service.py
│   │
│   ├── business/
│   │   ├── __init__.py
│   │   ├── carpooling_orchestrator.py
│   │   ├── clustering_service.py
│   │   └── greedy_optimizer.py
│   │
│   └── presentation/
│       ├── __init__.py
│       ├── cli.py
│       └── map_renderer.py
│
└── tests/
    ├── __init__.py
    ├── unit/
    │   └── __init__.py
    └── integration/
        └── __init__.py
```